

Terraform Lifecycle: Complete Flow + Our GKE Use Case

A practical guide covering what happens in every Terraform phase, followed by the same lifecycle mapped to GitHub Actions, GKE, Cloud SQL, Workload Identity, Argo CD, Spring Boot, Cloud SQL Auth Proxy, and Liquibase.

PART 1 - COMPLETE TERRAFORM LIFECYCLE

Phase	Stage	What exactly happens - 1 liner
0	Write configuration	You define the desired infrastructure in .tf files using resources, modules, variables, outputs, and provider configuration.
1	terraform init	Terraform initializes the working directory, downloads required providers/modules, and initializes the backend.
2	terraform validate	Terraform checks that the configuration is syntactically and structurally valid.
3	terraform plan	Terraform compares desired configuration with state and remote infrastructure, builds dependencies, and calculates proposed changes.
4	terraform apply	Terraform executes the approved changes through the appropriate provider APIs and updates state.
5	State management	Terraform records managed resources and important attributes in state so future plans can determine what changed.
6	Change -> plan -> apply	When configuration changes, Terraform calculates only the required create, update, replace, or destroy operations.
7	terraform destroy	Terraform removes the infrastructure it manages, following resource dependencies.

Where providers fit

Provider = the plugin Terraform uses to communicate with an external platform/API. In our case, the Google provider translates Terraform resource operations into Google Cloud API calls.

Terraform -> Google Provider -> Google Cloud API -> GKE / Cloud SQL / IAM

Resource vs Module

Resource: one infrastructure object, such as a GKE cluster or Cloud SQL instance.

Module: reusable Terraform configuration containing resources, variables, and outputs.

Provider: the mechanism Terraform uses to communicate with the target platform.

PART 2 - THE SAME LIFECYCLE IN OUR USE CASE

Target architecture: GitHub Actions + Terraform + GKE + Cloud SQL PostgreSQL + Workload Identity + Argo CD + Spring Boot + Cloud SQL Auth Proxy sidecar + Liquibase.

Phase	Our use case	Exactly what happens - 1 liner
0	Configuration	Terraform code defines the ephemeral environment: network, GKE, Cloud SQL, IAM, and Workload Identity.
1	terraform init	GitHub Actions initializes Terraform and downloads the Google provider, modules, and backend configuration.
2	terraform validate	The pipeline checks that the Terraform configuration is syntactically and structurally valid before planning.
3	terraform plan	Terraform reads configuration/state and queries GCP as needed, then calculates what must be created or changed for the PR environment.
4	terraform apply	Terraform uses the Google provider to create/update GCP infrastructure such as VPC, GKE, Cloud SQL, IAM, and Workload Identity.
5	State	Terraform records the created infrastructure in remote state so the next run knows what already exists and what has changed.
6	Argo CD	After infrastructure is ready, Argo CD reconciles Kubernetes desired state and deploys the Spring Boot workload.
7	Liquibase	Liquibase applies pending database schema migrations to Cloud SQL; Terraform itself does not manage application tables.
8	Change / destroy	Infrastructure changes go through plan/apply; when the ephemeral environment is no longer needed, destroy removes Terraform-managed infrastructure.

Parallel view: generic Terraform vs our use case

Terraform phase	Generic: what Terraform does	Our use case: what happens
Write configuration	Define desired infrastructure	Define ephemeral GKE + Cloud SQL + network + IAM + Workload Identity
init	Install providers/modules; initialize backend	Google provider + Terraform modules/backend become available in GitHub Actions
validate	Check configuration structure	Catch Terraform configuration errors before infrastructure changes
plan	Compare desired vs current and calculate changes	Determine whether PR environment needs VPC/GKE/Cloud SQL/IAM creation or updates
apply	Call provider APIs and make changes	Google provider calls GCP APIs to provision/update infrastructure
state	Record managed resource information	Remote state records the ephemeral environment resources for future runs
Application deployment	Not inherently Terraform's job	Argo CD deploys Spring Boot into GKE
Database schema	Not Terraform's primary responsibility	Liquibase applies schema migrations inside Cloud SQL
destroy	Remove managed infrastructure	Remove the PR's ephemeral GKE/Cloud SQL/network/IAM resources when no longer needed

END-TO-END PARALLEL DIAGRAM - OUR USE CASE

Generic Terraform lifecycle

Write configuration -> init -> validate -> plan -> apply -> state -> change/re-plan/re-apply -> destroy

What happens in our environment

```

  GitHub PR
    |
    v
  GitHub Actions
    |
    +--> terraform init --> Google provider + modules + backend ready
    |
    +--> terraform validate --> Terraform configuration checked
    |
    +--> terraform plan --> Desired state vs state/GCP -> dependency graph -> proposed changes
    |
    +--> terraform apply --> Google provider -> Google Cloud APIs
    |
    v
  GCP infrastructure
    |
    +--> VPC / Network
    +--> GKE
    +--> Cloud SQL PostgreSQL
    +--> IAM + Workload Identity
    |
    v
  Infrastructure ready
    |
    v
  Argo CD -> Spring Boot deployment in GKE
    |
    v
  Spring Boot Pod + Cloud SQL Auth Proxy sidecar
    |
    v
  Cloud SQL PostgreSQL
    |
    v
  Liquibase -> database schema migrations
```

Important boundaries

Tool	Primary responsibility
Terraform	Infrastructure: GKE, Cloud SQL, networking, IAM, Workload Identity, and related GCP resources.
Google Provider	Makes the GCP API calls requested by Terraform resources.
Argo CD	Application deployment and reconciliation in Kubernetes.
Cloud SQL Auth Proxy	Secure connection path from the application pod to Cloud SQL; used as a sidecar in the pod.
Liquibase	Database schema versioning and migrations.
Istio / Envoy	Not required for the current single Spring Boot service unless advanced service-mesh requirements appear.

Interview summary: Terraform manages infrastructure through providers. Init prepares Terraform, validate checks configuration, plan determines the delta and dependencies, apply makes the changes, state records what is managed, repeated plan/apply handles future changes, and destroy removes the managed infrastructure. In our architecture, Terraform stops at infrastructure; Argo CD handles application deployment and Liquibase handles database schema changes.